# Smooth Drift Analysis of Lin-UCB Algorithm Variants Project

Jelle Hendriks and Sulaf Al Jabal
Introduction to Sequential Decision Making - CDS DS 592
Prof. Aldo Pacchiano
T.F. Yichen Song

## Introduction

Many applications of modern life, such as recommendation systems in social media applications, rely on sequential decision-making under uncertainty. The framework used in stochastic linear bandit settings models these modern problems by assuming that rewards are linear in a context vector with $\theta^*$ acting as our unknown parameter. Algorithms such as Lin-UCB learn $\theta^*$, but assume that it is fixed. This assumption does not hold in reality, if we are to use the example of user social media content preferences as our primary use-case within this paper, then as a user explores content in a social media platform, we would expect some form of change in their preferences over time. It is then expected that algorithms which make use of this stationary assumption in the parameter $\theta$ would accumulate linear regret over time as user preferences change faster than the algorithms can adapt to said change.

Many ideas have been proposed to handle this non-stationary environment. Methods like dLinUCB, proposed by researchers Wu et al., utilize change-point detection methods, these test whether the environment has shifted (change in $\theta^*$) and restart estimation when a change has been detected. Whilst this algorithm shows promise in being able to adapt to different true values of $\theta^*$, we suspect that it will perform poorly under the context of social media user preference, as this kind of change in preference is usually a slow, and gradual one. We suspect that dLinUCB will have difficulty identifying clear timesteps in which to 'reset,' and will therefore accumulate more regret than other algorithms.

Other methods, such as SW-LinUCB [**?**], avoid detection entirely, and thus are deemed to be passive adaptation methods; this algorithm uses a sliding window to the most recent $\tau$ observations as a means of elimination. This could prove to mimic the behaviors of users online. If a user had an old hobby that they no longer observe, and replaces it with a new hobby, we feel that the user might prefer watching videos related to their *current* hobby instead of videos related to their older hobby.

The D-LinUCB [**?**] algorithm makes use of exponential discounting using a factor of $\gamma$ to smoothly forget/down weight old data. In their paper, Russac et al., show that the D-LinUCB algorithm achieves a dynamic regret bound of $O(d^{2/3}B_T^{1/3}T^{2/3})$. This bound is minimax optimal where the total variation of $\theta^*$ is captured through $B_T$.

However, the evaluation of the algorithm by Russac et al. primarily focuses on suddenly-changing environments, where $\theta^*$ jumps between fixed values at discrete breakpoints. If we are to use our social media example where $\theta^*$ represents a user's preferences over time, as mentioned earlier, the introduction of sharp-changing environments is in contrast with the reality: most people's likes and dislikes don't change at set times.

# Background

## Linear Bandits

In the stochastic linear bandit problem, a learner at every round, t, selects an action $a_t$ from a set of actions $A_t \subset R^d$. The learner then observes a reward vector $X_t =< \theta^*, a_t >$, where $\theta^* \in R^d$ is an unknown parameter and $\eta_t$ is sub-gaussian noise. The goal of the learner is to minimize the cumulative regret incurred, always relative to the to the best action played. LinUCB manages to solve this issue via regularized least-squares estimate $\theta_t = V_t^{-1} b_t$. Where $V_t = \lambda I + \Sigma a_s a_s^T$, and $b_t = \Sigma X_s a_s$. By selecting actions optimistically using $UCB(a) =< \hat{\theta}_t, a > + \beta_t ||a||_{V_t^{-1}}$. UCB achieves a regret bound of $O(d\sqrt{T})$ in the stationary environment.

## Non-stationary bandits

When $\theta^*$ is allowed to change over time, the metric we are tracking becomes the dynamic regret across all timesteps: $R_T = \Sigma_{t=1}^{T}(\max_a < \theta_t, a > - < \theta_t, a_t >)$. The difficulty of the problem is then characterized by the total variation: $B_T = \Sigma_{t=1}^{T-1} ||\theta_{t+1} - \theta_t||$. The textbook by Lattimore and Szepesvári [2020] use the discounted UCB and sliding window UCB for finite-armed non-stationary bandits in chapter 31; here, they establish the key principle that forgetting old data an appropriate rates allows for sublinear dynamic regret when $B_T$ grows sub-linearly in $T$.

## Proposed Evaluation

In this project, we will evaluate Lin-UCB, as well as its variants, dLinUCB, D-LinUCB, SW-LinUCB, on smooth drift functions not present in the original papers presented earlier. We will use the theoretically optimal tuning formulas that were derived by Russac et al. [2019], $\gamma^* = 1 - (B_T/dT)^{2/3}$ for D-LinUCB and $\tau^* = (dT/B_T)^{2/3}$ for SW-LinUCB. These two equations were established under worst-case assumptions. We will test whether they remain effective when the drift functions have known structure, e.g. linear or sinusoidal patterns.

# Algorithms

## LinUCB

We use the LinUCB with Disjoint Linear Models originally proposed by Lihong Li et al (2012). This assumes the expected payout of any arm is linear in its d-dimensional feature $x_{t,a}$ with some constant unknown coefficient vector to be estimated. It is similar to regular UCB models with always choosing the option with the highest UCB. It is still valid if input features are non-stationary and does not make that an assumption.

## D-LinUCB

Discounted least-squared Linear UCB, first presented by Russac et al. (and the basis of this report), is a variation of the Lin-UCB algorithm by discounting older observations. The algorithm uses weight $\gamma^{t-s}$ for observations $s$ at time $t$. This design matrix produced is as such: $V_t = \lambda I + \Sigma \gamma^{t-s} a_s a_s^T$
The estimator $\hat{\theta}_t = V_T^{-1} \Sigma \gamma^{t-s} X_s a_s$.
The weighted least squares deviations produces a confidence set containing a manipulated covariance term: $V_t^{-1} V_t^2 V_t^{-1}$. Where $V_t^2$ uses $\gamma^2$ instead of $\gamma$, the weights are squared in the variance of $\hat{\theta}$ and $\theta^*$ . The confidence width $\beta_t$ then depends on the the discounted dimension instead of the entire, raw dimension; this prevents the confidence set from growing with $t$. The Corollary 1 presented in the paper gives $\gamma^* = 1 - (B_T/dT)^{2/3}$, and a corresponding regret bound

**Algorithm 1:** LinUCB with Disjoint Linear Models

[1] **Input:** $\alpha \in \mathbb{R}_+$
**for** $t = 1, 2, 3, \ldots, T$ **do**
  Observe features of all arms $a \in \mathcal{A}_t$: $\mathbf{x}_{t,a} \in \mathbb{R}^d$
  **for** *all* $a \in \mathcal{A}_t$ **do**
    **if** *a is new* **then**
      $\mathbf{A}_a \leftarrow \mathbf{I}_d$ $d$-dimensional identity matrix
      $\mathbf{b}_a \leftarrow \mathbf{0}_{d\times 1}$ $d$-dimensional zero vector
    $\hat{\boldsymbol{\theta}}_a \leftarrow \mathbf{A}_a^{-1}\mathbf{b}_a$ $p_{t,a} \leftarrow \hat{\boldsymbol{\theta}}_a^\top \mathbf{x}_{t,a} + \alpha\sqrt{\mathbf{x}_{t,a}^\top \mathbf{A}_a^{-1}\mathbf{x}_{t,a}}$
  Choose arm $a_t = \arg\max_{a\in\mathcal{A}_t} p_{t,a}$ with ties broken arbitrarily, and observe a real-valued payoff $r_t$

  $\mathbf{A}_{a_t} \leftarrow \mathbf{A}_{a_t} + \mathbf{x}_{t,a_t}\mathbf{x}_{t,a_t}^\top$
  $\mathbf{b}_{a_t} \leftarrow \mathbf{b}_{a_t} + r_t\mathbf{x}_{t,a_t}$

of $O(d^{2/3}B_T^{1/3}T^{2/3})$

**Algorithm 2:** D-LinUCB

**Input:** Probability $\delta$, sub-gaussianity constant $\sigma$, dimension $d$, regularization $\lambda$, upper bound for actions $L$, upper bound for parameters $S$, discount factor $\gamma$.
**Initialization:** $b = 0_{\mathbb{R}^d}$, $V = \lambda I_d$, $\widetilde{V} = \lambda I_d$, $\hat{\theta} = 0_{\mathbb{R}^d}$
**for** $t \geq 1$ **do**
  Receive $\mathcal{A}_t$, compute $\beta_{t-1} = \sqrt{\lambda}S + \sigma\sqrt{2\log\left(\frac{1}{\delta}\right) + d\log\left(1 + \frac{L^2(1-\gamma^{2(t-1)})}{\lambda d(1-\gamma^2)}\right)}$
  **for** $a \in \mathcal{A}_t$ **do**
    Compute $\mathrm{UCB}(a) = a^\top\hat{\theta} + \beta_{t-1}\sqrt{a^\top V^{-1}\widetilde{V}V^{-1}a}$
  $A_t = \arg\max_a(\mathrm{UCB}(a))$
  **Play action** $A_t$ **and receive reward** $X_t$
  **Updating phase:**
  $V = \gamma V + A_tA_t^\top + (1-\gamma)\lambda I_d$
  $\widetilde{V} = \gamma^2\widetilde{V} + A_tA_t^\top + (1-\gamma^2)\lambda I_d$
  $b = \gamma b + X_tA_t$, $\hat{\theta} = V^{-1}b$

## SW-LinUCB

Cheung et al.'s [2019] variation is called Sliding Window Linear UCB algorithm. This a different approach by restricting the least-squares estimator to the last $\tau$ observations. This provides a hard cutoff as opposed to smooth forgetting. The optimal window is $\tau^* = (dT/B_T)^{2/3}$. This is a regret rate similar to D-LinUCB.

**Algorithm 3:** SW-UCB algorithm

**Input:** Sliding window size $w$, dimension $d$, variance proxy of the noise terms $R$, upper bound of all the actions' $\ell_2$ norms $L$, upper bound of all the $\theta_t$'s $\ell_2$ norms $S$, and regularization constant $\lambda$.

**Initialization:** $V_0 \leftarrow \lambda I$

**for** $t = 1, \ldots, T$ **do**

$\hat{\theta}_t \leftarrow V_{t-1}^{-1} \left( \sum_{s=1 \vee (t-w)}^{t-1} X_s Y_s \right)$

$X_t \leftarrow \text{argmax}_{x \in D_t} \left\{ x^\top \hat{\theta}_t + \|x\|_{V_{t-1}^{-1}} \left[ R \sqrt{d \ln \left( \frac{1 + wL^2/\lambda}{\delta} \right)} + \sqrt{\lambda} S \right] \right\}$

$Y_t \leftarrow \langle X_t, \theta_t \rangle + \eta_t$

$V_t \leftarrow \lambda I + \sum_{s=1 \vee (t-w+1)}^{t} X_s X_s^\top$

## dLinUCB

Wu et al. [2018] propose a difference algorithm which uses an approach that utilizes active change-point detection. It consists of a master algorithm which watches over a set of LinUCB 'slaves', each initialized at a different time step. The master tracks every slave's 'badness', the fraction of recent predictions. This badness is a fraction of recent predictions where the error exceeded the confidence bound. The master then discards the slaves who achieve a badness score that is too high. The master then spawns a new slave when a change in $\theta$ (context: change in Instagram reals video preferences) occurs. This whole process then repeats. This kind of approach is appropriate for sharp , and infrequent changes in theta , but struggles with gradual changes in theta since there is no discrete change point which triggers detection.

**Algorithm 4:** Dynamic Linear UCB (dLinUCB)

**Input:** $\lambda > 0$, $\tau > 0$, $\delta_1, \delta_2 \in (0,1)$, $\tilde{\delta}_1 \in [0, \delta_1]$
**Initialize:** Maintain a set of slave models $\mathcal{M}_t$ with $\mathcal{M}_1 = \{m_1\}$, initialize $m_1$: $\mathbf{A}_1(m_1) = \lambda \mathbf{I}$, $\mathbf{b}_1(m_1) = \mathbf{0}$, $\hat{\theta}_1(m_1) = \mathbf{0}$; and initialize the 'badness' statistics of it: $\hat{e}_1(m_1) = 0$, $d_1(m_1) = 0$
**for** $t = 1$ *to* $T$ **do**
  Choose a slave model from the active slave model set $\tilde{m}_t = \arg\min_{m \in \mathcal{M}_t} \left( \hat{e}_t(m) - \sqrt{\ln \tau} \times d_t(m) \right)$
  Observe candidate arm pool $\mathcal{A}_t$, with $\mathbf{x}_a \in \mathbb{R}^d$ for $\forall a \in \mathcal{A}_t$
  Take action $a_t = \arg\max_{a \in \mathcal{A}_t} \left( \mathbf{x}_a^\top \hat{\theta}_t(\tilde{m}_t) + B_t(\tilde{m}_t, a) \right)$, in which $B_t(\tilde{m}_t, a)$ is defined in Eq (3)
  Observe payoff $r_{a_t}$
  Set CreatNewFlag = True
  **for** $m \in \mathcal{M}_t$ **do**
    $e_t(m) = \mathbb{1}\{|\hat{r}_t(m) - r_t| > B_t(m, a) + \epsilon\}$, where $\hat{r}_t(m) = \mathbf{x}_{a_t}^\top \hat{\theta}_t(m)$ and $\epsilon = \sqrt{2}\sigma \mathrm{erf}^{-1}(\delta_1 - 1)$
    **if** $e_t(m) = 0$ **then**
      Update slave model: $\mathbf{A}_{t+1}(m) = \mathbf{A}_t(m) + \mathbf{x}_{a_t}\mathbf{x}_{a_t}^\top$, $\mathbf{b}_{t+1}(m) = \mathbf{b}_t(m) + \mathbf{x}_{a_t} r_t$, $\hat{\theta}_{t+1} = \mathbf{A}_{t+1}^{-1}(m)\mathbf{b}_{t+1}(m)$
    $\tilde{\tau}(m) = \min\{t - t_m, \tau\}$, where $t_m$ is when $m$ was created
    Update 'badness' $\hat{e}_t(m) = \frac{\sum_{i=t-\tilde{\tau}}^{t} e_i(m)}{\tilde{\tau}(m)}$, $d_t(m) = \sqrt{\frac{\ln 1/\delta_2}{2\tilde{\tau}(m)}}$
    **if** $\hat{e}_t(m) < \tilde{\delta}_1 + d_t(m)$ **then**
      Set CreatNewFlag = False
    **else if** $\hat{e}_t(m) \geq \delta_1 + d_t(m)$ **then**
      Discard slave model $m$: $\mathcal{M}_{t+1} = \mathcal{M}_t - m$
  **if** *CreatNewFlag or* $\mathcal{M}_t = \emptyset$ **then**
    Create a new slave model $m_t$: $\mathcal{M}_{t+1} = \mathcal{M}_t + m_t$
    Initialize $m_t$: $\mathbf{A}_t(m_t) = \lambda \mathbf{I}$, $\mathbf{b}_t(m_t) = \mathbf{0}$, $\hat{\theta}_t(m_t) = \mathbf{0}$
    Initialize 'badness' statistics of $m_t$: $\hat{e}_t(m_t) = 0$, $d_t(m_t) = 0$

# Experiments

For each drift type, we compute the total variation $B_T = \Sigma ||\theta^*_{t+1} - \theta^*_t||$ directly from the drift function over the full horizon $T = 6000$. This quantity measures the total amount of non-stationary (lack of change in $\theta^*$) in the environment, and is used to set the hyperparameters of the four algorithms explored in this paper. D-LinUCB's discount factor is set to $\gamma^* = 1 - (\frac{B_T}{dT})^{2/3}$ following the Corollary 1 of Russac et al. [2019]. SW-LinUCB's window size is set to $\tau^* = (\frac{dT}{B_T})^{2/3}$, while dLinUCB uses $\tau^*$ as its 'badness tracking' window. LinUCB, on the other hand, has no forgetting mechanism, and therefore requires no drift-dependent tuning; it serves as our stationary baseline to compare to. All the regret values averaged over 100 independent runs.

## Abrupt Drifting Function

For this work we used a two different smooth non-stationary environments and a baseline of the abruptly changing environment used by [**?**]. For our new environments, we decided to use a linear model and a sinusoidal model both with two different speeds at which they shifted. We observed the estimated mean reward parameter by the different models and first attempted to

reproduce [?] except with multiple shifts in model parameter. They never indicated the outlook of how the estimated parameter changes within the model but we had a noticeably different result from their model even with only one abrupt shift. Their models did similar before the first shift in parameter but did significantly worse after the first shift. Except for the relagure LinUCB model which did similarly bad but for some reason their dLinUCB did not learn and their others did not learn sufficiently. As can be observed, our models are not significantly affected by the abrupt changes at least at two parameters and are able to rapidly adapt to the changes with barely a shift in regret. Unlike in the original paper, we found that the D-LinUCB is slightly worse in terms of regret when under repeated abrupt changes than the SW-LinUCB model as it is not able to react as quickly.

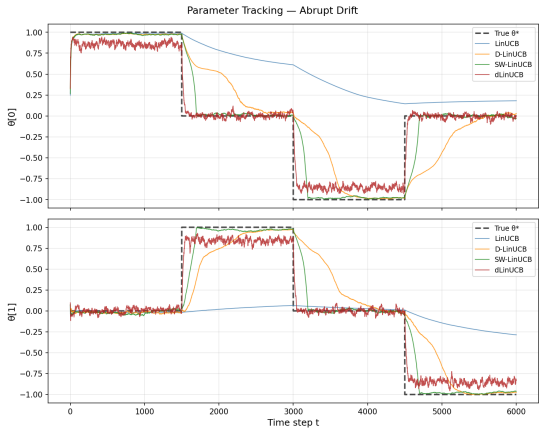


Figure 1: The estimated parameters over time in the different models as the actual abruptly changes

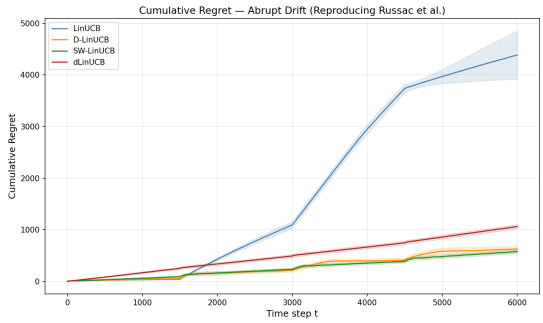


Figure 2: The average regret over time in the different models as the actual abruptly changes

## Linear Drifting Function

We shall start with the more basic slow linear drifting function. Within this system one of the arms linearly decreases from an expected reward of 1 to slowly go to 0.5 over the course while the other increases over time from an expected reward of 0 to go to 0.5. This means that the entire time the best result is to pick arm 0 every time but the regret for picking arm 1 decreases as it continues. This model is mainly there to observe the way the model experiments as the two arms get closer and closer. From this it can be seen that the dLinUCB model experiments constantly with no change in regret from changing difference in reward. In comparison, the other models do well to varying degrees. it seems that SW-LinUCB experiments the most as it forgets what might have previously been the best result whereas D-LinUCB experiments less than even the regular LinUCB and continuously decides the best arm to pick at any time. Now we do the same observations for the Fast Linear Drifting function where the expected reward for arm 0 goes from 1 to -1 and the expected reward for arm 1 is stationary at 0. This results in the model needing to decide to switch to picking the other arm half way through the process. This can't be done by the LinUCB model and thus it shoots up a bit over halfway through. The other models do not have significant changes over time and are just linear except for when it is unable to get regret halfway through. Although SW-LinUCB and D-LinUCB do similarly it seems that D-LinUCB does consistently better for these environments. Interestingly though when observing the estimated parameter it seems that SW-LinUCB has a better estimate of what is the best arm as it hovers closer to the actual estimated parameter.

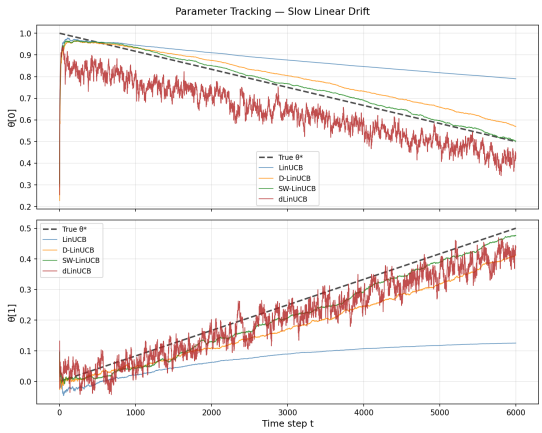


Figure 3: The estimated parameters over time in the different models as the actual slowly shifts linearly.

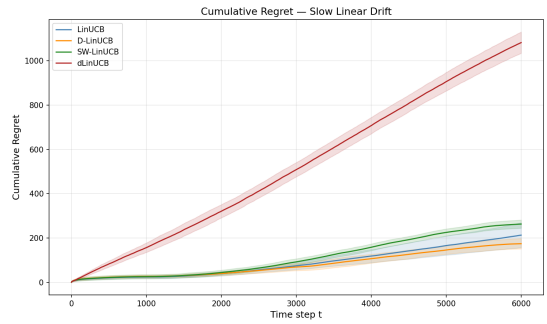


Figure 4: The average regret over time in the different models as the actual parameter slowly changes linearly

.

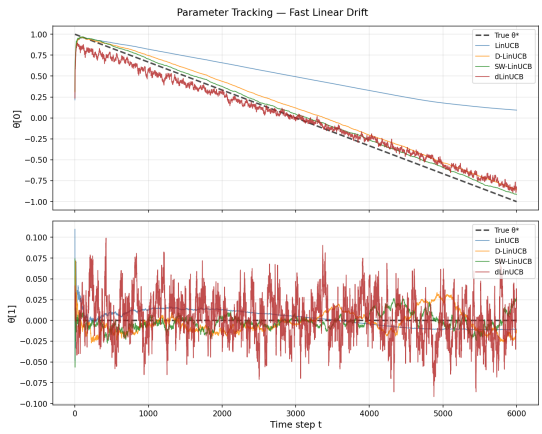


Figure 5: The estimated parameters over time in the different models as the actual swiftly changes linearly.

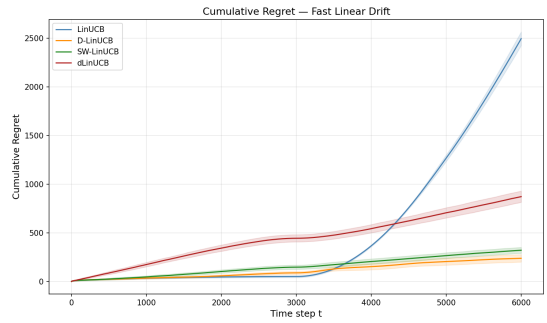


Figure 6: The average regret over time in the different models as the actual parameter rapidly changes linearly.

## Sinusoidal Drifting Function

We shall start with the slow function again but this time for a sinusoidal function. This time, the sinusoidal function has arm 0 as a cosine function starting from a reward of 0.5 and going down to -0.5 and back to 0.5 over the course of the tests. And arm 1 instead starts as a sine function with the same amplitude and period. The result for which arm is best changes multiple times even in this slower environment. The Lin-UCB model still struggles with these changes but all the other models seem to still stay within a mostly linear regret. Now observing the fast sinusoidal function which is setup to instead have the period pass six times with all other factors constant. Again the Lin-UCB model struggles as expected but the other models are mostly linear. It seems that dLinUCB has relatively constant regret no matter the rate at which the environment changes but it does worse than the other two remaining models. Both SW-LinUCB and D-LinUCB do similarly but D-LinUCB still does consistently better.

## Summary of Results

Overall it seems D-LinUCB seems to be the best model for smooth drifting functions even if not as fast reacting for abrupt drifting functions. dLinUCB seems to be relatively constant with its

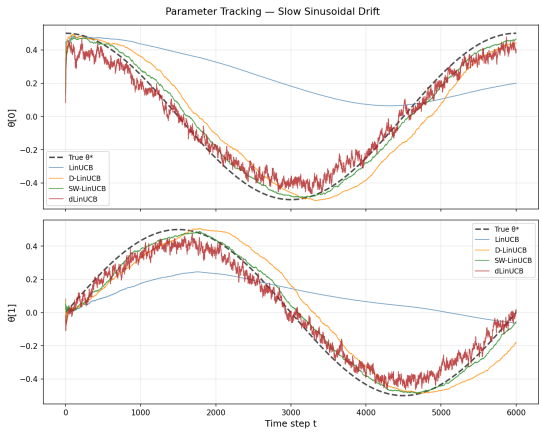


Figure 7: The estimated parameters over time in the different models as they slowly shift sinusoidally.

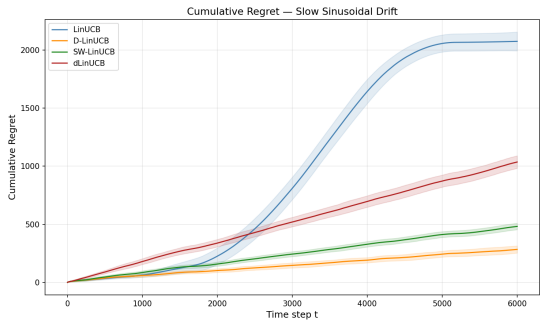


Figure 8: The average regret over time in the different models as the parameter slowly shifts sinusoidally

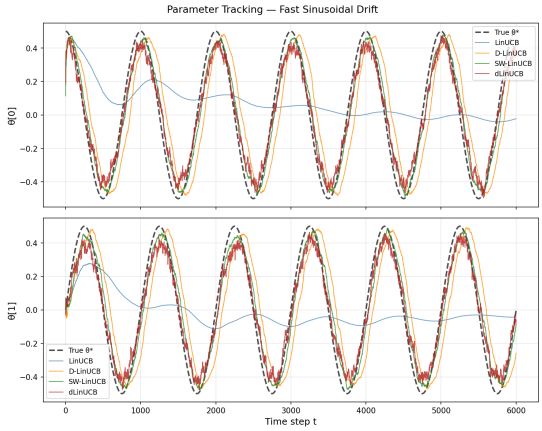


Figure 9: The estimated parameters over time in the different models as the actual parameter shifts sinusoidally

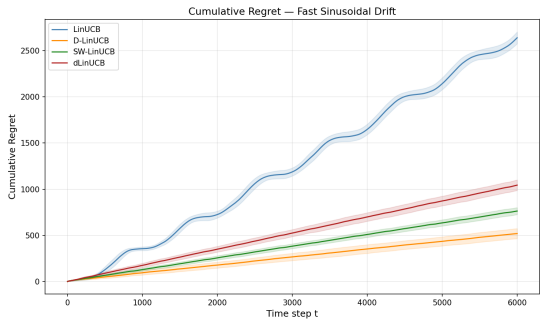


Figure 10: The average regret over time in the different models as the parameter rapidly shifts sinusoidally

regret but consistently worse than the other two models due to its high variance in choices, it is possible it could be better in very particular models though. LinUCB is best on completely stationary environments of them but at any change or drift it starts to struggle as expected. SW-LinUCB is a bit of a middle ground with D-LinUCB that could adapt faster when there are a lot of abrupt and repeated changes but does worse on any slower or smoother model. Overall for any model that could have changes in results both SW-LinUCB and D-LinUCB are valid in varying situations and either can be used.

## Conclusion

The D-LinUCB and SW-LinUCB model both work well as adaptive models for varying drifting functions. Additional future work could be observing the time to run for both of the models as it seemed during our testing that the SW-LinUCB was very slow and took the most time out of any of them by a significant amount. We could not verify entirely that this was the one so further testing on time could be useful for larger datasets. This could indicate that it might not be feasible to use these adaptive models if they are significantly slower for large datasets.

Additional work would be to find other comparable adaptive models that might have stronger results or different models to be able to compare our results better to other adaptive models

Table 1: Comparison of Algorithm Regret Across Different Drift Types

| **Drift Type** | **LinUCB** | **D-LinUCB** | **SW-LinUCB** | **dLinUCB** |
|---|---|---|---|---|
| Abrupt | 4400 | **600** | 660 | 1000 |
| Slow Linear | 200 | **195** | 240 | 1080 |
| Fast Linear | 2500 | **250** | 330 | 880 |
| Slow Sinusoidal | 2100 | **300** | 500 | 1010 |
| Fast Sinusoidal | 2700 | **500** | 750 | 1000 |

instead of the mostly stationary models that are currently there. An interesting feature of our work is that for us the dLinUCB model did not do as bad as they did in the paper we were emulating [**?**]. They seemed to indicate it as an almost stationary model but for us it varies quite well to changes in the data even if worse than the specific models. Thus it seems quite plausible that with more tuning to it, it could have been better as we did not investigate it at all.